



nextwork.org

VPC Peering



Mohammed Dawoud Mota

Accept VPC peering connection request [Info](#) ✕

Are you sure you want to accept this VPC peering connection request? (pcx-0e0f24eff867d616e / VPC 1 <-> VPC 2)

Requester VPC vpc-097719aea34559033 / NextWork-1-vpc	Accepter VPC vpc-022d312ba1befb46d / NextWork-2-vpc	Requester CIDRs 10.1.0.0/16
Accepter CIDRs -	Requester Region N. Virginia (us-east-1)	Accepter Region N. Virginia (us-east-1)
Requester owner ID 289654514139(This account)	Accepter owner ID 289654514139(This account)	

[Cancel](#) [Accept request](#)



Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a virtual private cloud that lets you create and control a secure network within AWS. It is useful because it allows you to isolate resources, manage traffic, and enhance security for your applications.

How I used Amazon VPC in this project

I created two VPCs and connected them using a VPC peering connection so they could communicate privately. I also configured route tables and security settings to enable secure data transfer between them.

One thing I didn't expect in this project was...

I didn't expect to face connection errors when trying to use EC2 Instance Connect. It helped me understand how important public IPs and correct security group settings are for successful connections.



This project took me...

This project took me 40 minutes to complete.



In the first part of my project...

Step 1 - Set up my VPC

In this step, I will create two VPCs from scratch. I will do that by using the Visual VPC resource map to be able to create the VPC very fast.

Step 2 - Create a Peering Connection

In this step, I will set up a peering connection between the two VPCs I have created. This will allow the two VPCs to connect and communicate with each other.

Step 3 - Update Route Tables

In this step, I will set up the route tables to allow for traffic coming from VPC 1 to VPC 2. And I will set up a way for traffic coming from VPC 2 to VPC 1.

Step 4 - Launch EC2 Instances

In this step, I will launch an EC2 instance in each VPC so I can use them to test the VPC peering connection.



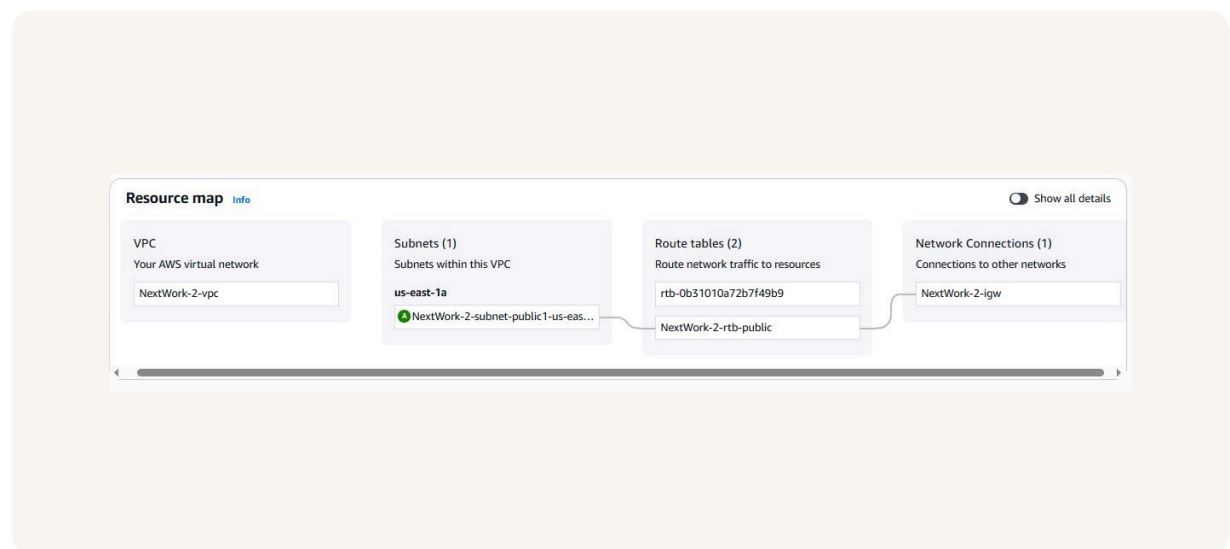
Multi-VPC Architecture

I started my project by launching two VPCs each is one AZ and each has on public subnet. So a total for 2 public subnets across 2 VPCs.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16. They have to be unique so the IP addresses of their resources don't overlap. Having overlapping IP addresses could cause routing conflicts and connectivity issues.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances because we can use EC2 instance connect, where AWS stores the key for us, and we can connect to the instance without having to SSH into the instance ourselves.





VPC Peering

A VPC peering connection is a direct connection between two VPCs. It allows the VPCs and their resources to route traffic between them using the private IPs.

VPCs would use peering connections so that data can be transferred between VPCs without going through the public internet.

A requester is the VPC that initiates a peering connection; they will send the invitation to connect. The Acceptor is the VPC that receives a peering connection request. They can either accept or decline the invitation.

Select another VPC to peer with

Account

☒ My account
☐ Another account

Region

☒ This Region (us-east-1)
☐ Another Region

VPC ID (Acceptor)

vpc-022d312ba1befb46d (NextWork-2-vpc) ▼

VPC CIDRs for vpc-022d312ba1befb46d (NextWork-2-vpc)

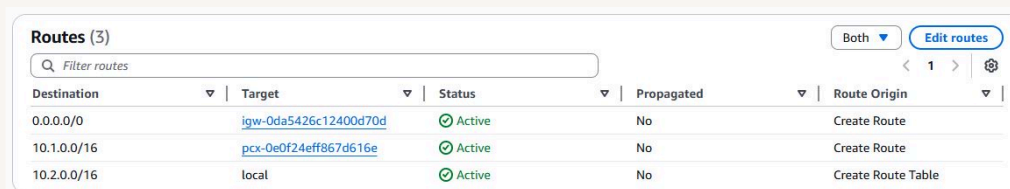
CIDR	Status	Status reason
10.2.0.0/16	✔ Associated	-



Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because VPC 1 needs to know how to get the resources in VPC 2. The route table will direct traffic that will be bound to VPC 2.

My VPCs' new routes have a destination of 10.1.0.0/16 and 10.2.0.0/16. The routes' target was VPC 1 and VPC 2.



The screenshot shows the 'Routes (3)' section of the AWS Management Console. It includes a search bar, a 'Both' dropdown, and an 'Edit routes' button. Below is a table with three routes. The first route is for 0.0.0.0/0 with target igw-0da5426c12400d70d. The second route is for 10.1.0.0/16 with target pcx-0e0f24eff867d616e. The third route is for 10.2.0.0/16 with target local. All routes are 'Active' and 'No' propagated. The 'Route Origin' column shows 'Create Route' for the first two and 'Create Route Table' for the third.

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0da5426c12400d70d	Active	No	Create Route
10.1.0.0/16	pcx-0e0f24eff867d616e	Active	No	Create Route
10.2.0.0/16	local	Active	No	Create Route Table



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will use the EC2 instance to connect to the first EC2 instance and fix any connection issues that arise.

Step 6 - Connect to EC2 Instance 1

In this step, I will use the EC2 instance to connect to the first Instance. And fix any issues or errors that come up while connecting.

Step 7 - Test VPC Peering

In this step, I will get the first instance to send test messages to the second instance. And then I will fix any issues that come up with the connection between the two instances.



Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to be able to connect to the first EC2 instance.

I was stopped from using EC2 Instance Connect because there was no public IP for the instance, because we had the setting disabled for automatically assigning public IPs.

Connect Info
Connect to an instance using the browser-based client.

EC2 Instance Connect

Session Manager

SSH client

EC2 serial console

No public IPv4 or IPv6 address assigned

With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect. Alternatively, you can try connecting using [EC2 Instance Connect Endpoint](#).

Instance ID

t-0f44f282bb6700ad9

(Instance - NextWork VPC 1)

Connection type

☒ Connect using a Public IP

Connect using a public IPv4 or IPv6 address

☐ Connect using a Private IP

Connect using a private IP address and a VPC endpoint

☐ Public IPv4 address

☐ IPv6 address

Username

Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.

ec2-user

X

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.



Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static IPv4 addresses that get allocated to your AWS account.

Associating an Elastic IP address resolved the error because it gave the instance a public IP that we can connect to through the EC2 instance connect.

Allocate Elastic IP address info

Elastic IP address settings info

Public IPv4 address pool

- ☒ Amazon's pool of IPv4 addresses
- ☐ Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) [Learn more](#)
- ☐ Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer-owned pools found) [Learn more](#)
- ☐ Allocate using an IPv4 IPAM pool (option disabled because no public IPv4 IPAM pools with AWS service as EC2 were found)

Network border group info

Q us-east-1 X

Global static IP addresses

AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

[Create accelerator](#)

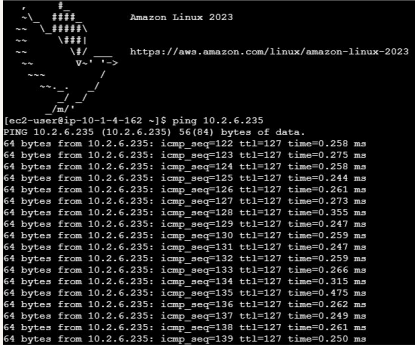


Troubleshooting ping issues

To test VPC peering, I ran the command ping. Ping is a common computer network tool used to check whether your computer can communicate with another computer or device on a network.

A successful ping test would validate my VPC peering connection because it shows the two instances are connecting and are able to communicate with each other.

I had to update my second EC2 instance's security group because the ping command was not working. I added a new rule that allowed ICMP traffic from VPC 1's CIDR block.



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-1-4-162 ~]$ ping 10.2.6.235
PING 10.2.6.235 (10.2.6.235): 56(84) bytes of data:
64 bytes from 10.2.6.235: icmp_seq=122 ttl=127 time=0.258 ms
64 bytes from 10.2.6.235: icmp_seq=123 ttl=127 time=0.275 ms
64 bytes from 10.2.6.235: icmp_seq=124 ttl=127 time=0.258 ms
64 bytes from 10.2.6.235: icmp_seq=125 ttl=127 time=0.244 ms
64 bytes from 10.2.6.235: icmp_seq=126 ttl=127 time=0.261 ms
64 bytes from 10.2.6.235: icmp_seq=127 ttl=127 time=0.273 ms
64 bytes from 10.2.6.235: icmp_seq=128 ttl=127 time=0.355 ms
64 bytes from 10.2.6.235: icmp_seq=129 ttl=127 time=0.247 ms
64 bytes from 10.2.6.235: icmp_seq=130 ttl=127 time=0.259 ms
64 bytes from 10.2.6.235: icmp_seq=131 ttl=127 time=0.247 ms
64 bytes from 10.2.6.235: icmp_seq=132 ttl=127 time=0.255 ms
64 bytes from 10.2.6.235: icmp_seq=133 ttl=127 time=0.266 ms
64 bytes from 10.2.6.235: icmp_seq=134 ttl=127 time=0.315 ms
64 bytes from 10.2.6.235: icmp_seq=135 ttl=127 time=0.475 ms
64 bytes from 10.2.6.235: icmp_seq=136 ttl=127 time=0.262 ms
64 bytes from 10.2.6.235: icmp_seq=137 ttl=127 time=0.249 ms
64 bytes from 10.2.6.235: icmp_seq=138 ttl=127 time=0.261 ms
64 bytes from 10.2.6.235: icmp_seq=139 ttl=127 time=0.250 ms
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

